```
.../app/controllers
.../app/helpers
.../app/mailers
.../app/models
.../app/controllers/concerns
.../app/models/concerns
.../test/mailers/previews
```

`autoload_paths` is computed and cached during the initialization process. The application needs to be restarted to reflect any changes in the directory structure.

6 Autoloading Algorithms

6.1 Relative References

A relative constant reference may appear in several places, for example, in

```
class PostsController < ApplicationController
  def index
    @posts = Post.all
  end
end
```

all three constant references are relative.

6.1.1 Constants after the `class` and `module` Keywords

Ruby performs a lookup for the constant that follows a `class` or `module` keyword because it needs to know if the class or module is going to be created or reopened.

If the constant is not defined at that point it is not considered to be a missing constant, autoloading is**not** triggered.

So, in the previous example, if `PostsController` is not defined when the file is interpreted Rails autoloading is not going to be triggered, Ruby will just define the controller.

6.1.2 Top-Level Constants

On the contrary, if `ApplicationController` is unknown, the constant is considered missing and an autoload is going to be attempted by Rails.

In order to load `ApplicationController`, Rails iterates over `autoload_paths`. First checks if `app/assets/application_controller.rb` exists. If it does not, which is normally the case, it continues and finds `app/controllers/application_controller.rb`.

If the file defines the constant `ApplicationController` all is fine, otherwise `LoadError` is raised:

```
unable to autoload constant ApplicationController, expected
<full path to application_controller.rb> to define it (LoadError)
```

Rails does not require the value of autoloaded constants to be a class or module object. For example, if the file `app/models/max_clients.rb` defines `MAX_CLIENTS = 100`autoloading `MAX_CLIENTS` works just fine.

6.1.3 Namespaces

Autoloading `ApplicationController` looks directly under the directories of `autoload_paths`because the nesting in that spot is empty. The situation of `Post` is different, the nesting in that line is `[PostsController]` and support for namespaces comes into play.

The basic idea is that given

```
module Admin
  class BaseController < ApplicationController
```